NS NEXUSSHIELD

PROBLEM STATEMENT 6 · ET AI HACKATHON

Detailed Project Document

AI for Digital Public Safety

Defeating Counterfeiting, Fraud & Digital Arrest Scams

Team: NexusAI

Members: Abinivesh M · Abhinivesh K · Akshaya M · Yazhini K

Track: Smart Cities / Public Safety / Digital Trust

Stack: Flutter (Android) · FastAPI · Google Gemini API · Railway

CONTENTS

Table of Contents

1. Executive Summary
2. Problem Statement & Context
3. Challenge Brief (PS6) & Coverage Map
4. Solution Overview
5. System Architecture
6. Feature Deep Dive: Digital Arrest Scam Detection
7. Feature Deep Dive: Counterfeit Currency Detection
8. Feature Deep Dive: Fraud Network Intelligence
9. Feature Deep Dive: Geospatial Crime Intelligence
10. Feature Deep Dive: Citizen Fraud Shield
11. Platform Layer (Auth, History, Family, Org, Admin)
12. Technology Stack
13. Scope & Auditability — Live vs. Roadmap
14. Business Impact & Scalability
15. Known Limitations & Next Steps
16. Conclusion

1 Executive Summary

NexusShield is a working, end-to-end AI-powered public safety platform built in response to Problem Statement 6: AI for Digital Public Safety — Defeating Counterfeiting, Fraud & Digital Arrest Scams. Rather than building a single point-solution, NexusShield implements four of the five directions the organisers explicitly suggested — digital arrest scam detection, counterfeit currency identification, fraud network graph intelligence, and geospatial crime intelligence — plus a fifth, citizen-facing layer (multi-channel fraud shield with conversational AI, family protection, and an enterprise/admin layer) that ties the whole platform together into a single coherent product rather than five disconnected demos.

The system is fully deployed and testable today: a Flutter Android application communicates with a FastAPI backend hosted on Railway, which in turn calls the Google Gemini API for vision and language analysis. Every feature described in this document, unless explicitly marked otherwise in Section 13, is live in the current build — not a mockup, wireframe, or planned feature.

At a glance

5 PS6-aligned feature pillars · Flutter Android app + FastAPI backend + Gemini AI · Deployed on Railway with a live, testable API · 10 India-specific scam categories · 5-language AI output (English, Tamil, Hindi, Kannada, Telugu) · Biometric app lock, JWT auth, Google Sign-In

2 Problem Statement & Context

India registered 1.14 million cybercrime complaints in 2023 — a 60% increase over 2022 — and the trajectory has continued to steepen. Within this, “digital arrest” scams have emerged as an especially damaging pattern: fraudsters impersonate CBI, ED, Customs, or police officials and trap victims in multi-day psychological hostage situations over video call, using spoofed numbers, AI-generated voices, and fake government portals. The Ministry of Home Affairs reported these scams defrauded citizens of over ■1,776 crore in just the first nine months of 2024 alone.

Separately, counterfeit currency remains a persistent threat. The RBI’s 2025 Annual Report flagged record seizures of Fake Indian Currency Notes (FICN), with high-denomination notes now printed at a quality sufficient to defeat manual detection in routine banking operations.

The structural problem underlying both threats is the same: law enforcement does not lack evidence after the fact — it lacks intelligence before mass victimisation occurs. Existing tools are reactive (victims report after losing money, by which point the scam ring has already moved to new numbers and UPI IDs) and siloed (bank fraud teams, telecom abuse teams, and cybercrime cells rarely share signals in real time). Most consumer-facing apps check exactly one thing — a link, or a phone number — when real scams span calls, messages, fake websites, and physical cash simultaneously.

Solving this requires convergence of financial transaction intelligence, communication network analysis, physical security (counterfeit detection), and real-time public safety coordination — exactly the kind of multi-source intelligence problem NexusShield was built to address.

3 Challenge Brief (PS6) & Coverage Map

The organisers' brief calls for an AI-powered Digital Public Safety Intelligence platform that equips law enforcement, financial institutions, and citizens with proactive tools to detect, disrupt, and respond to digital fraud networks, counterfeit currency circulation, and organised scam operations — shifting from reactive case investigation to predictive threat neutralisation. Five build directions were suggested as illustrative starting points. NexusShield's coverage of each is summarised below.

Feature	Status
Digital Arrest Scam Detection & Alerting	BUILT
Counterfeit Currency Identification Agent	BUILT
Fraud Network Graph Intelligence	BUILT
Geospatial Crime Pattern Intelligence	BUILT
Citizen Fraud Shield (multi-channel)	BUILT

All five suggested directions are represented in the current build. Sections 6 through 10 describe each in technical detail, including exactly what is implemented, what data flows through it, and which parts remain explicitly scoped as roadmap (see Section 13 for a full and honest breakdown).

4 Solution Overview

NexusShield is architected as a single platform with five feature pillars sitting on top of a shared authentication, history, and intelligence layer — not five separate apps stitched together. A citizen opens one app to check a suspicious message, scan a banknote, search a phone number against known fraud rings, or view a live crime hotspot map; every one of those actions writes to the same scan-history table and contributes, where verified, to the same shared scam database that every other feature reads from.

Pillar	What it does
Digital Arrest Scam Detection	Analyzes text, screenshots, URLs, and call transcripts for authority impersonation, urgency, and payment-pressure patterns.
Counterfeit Currency Detection	Gemini Vision assesses a photographed banknote's security thread, watermark, serial number, and print quality.
Fraud Network Intelligence	Graph-style traversal linking phone numbers, UPI IDs, websites, devices, and bank accounts across scam reports.
Geospatial Crime Intelligence	Live hotspot map (heatmap + pin views) built from geotagged scam reports, filterable by category.
Citizen Fraud Shield	Conversational AI copilot, family protection alerts, enterprise dashboard, and admin review panel.

5 System Architecture

The system is organised into four layers: a Flutter client, a network layer enforcing stateless token authentication over HTTPS, a modular FastAPI backend, and an AI + data layer. Each layer is described below, followed by the deployment model.

Client Layer — Flutter Android App

Built with Flutter (Dart), targeting Android with a codebase structured for straightforward iOS porting later. Screens are organised by feature: five core analyzer screens (screenshot, text, URL, call transcript, currency), a fraud network search screen, a hotspot map screen, a conversational copilot screen, and a full supporting set for auth, profile, settings, history, family protection, organization dashboard, and admin panel. State is managed via the Provider package, with a single central `ApiService` (extends ChangeNotifier) handling every HTTP call, token refresh, and the cached current-user profile object that the whole app reactively listens to.

Network Layer

All traffic runs over HTTPS. Every authenticated request carries a short-lived JWT access token (HS256, 30-minute expiry) in the Authorization header; a separate long-lived refresh token (30-day expiry, individually revocable) is used to mint new access tokens without requiring the user to log in again. This is fully stateless on the server side — no session affinity is required, which matters for horizontal scaling.

Backend Layer — FastAPI on Railway

The backend is a modular FastAPI application with one router per feature area: auth_router, profile_router, analysis_router (screenshot/text/URL/clipboard/copilot), call_router, currency_router, intel_router (hotspots + fraud network), history_router, reports_router, family_router, org_router, admin_router, dashboard_router, notifications_router, integrations_router, and report_pdf_router. Each router imports shared SQLAlchemy models and Pydantic schemas but owns its own endpoint logic, so features can be modified or scaled independently without cross-router coupling.

A deliberate engineering decision worth noting: analysis endpoints that call Gemini (analyze_screenshot, analyze_text, analyze_call, analyze_currency, copilot_chat) are declared as synchronous (plain def, not async def) route handlers. FastAPI automatically runs plain-def endpoints in a thread pool; declaring them async def while calling a blocking Gemini SDK call from inside them would freeze the entire single-threaded event loop for the duration of every AI call, stalling unrelated requests (e.g. the hotspot map or fraud network search) system-wide. This was identified and fixed during development.

AI + Data Layer

All AI calls are isolated in a single module, ai_engine.py, which wraps the Google Gemini API (gemini-2.5-flash by default, configurable via environment variable). This isolation means the underlying model or provider can be swapped without touching any router code. Before any AI call is made for text/URL

analysis, the request is first checked against a local Live Scam Database (scam_db.py) of known-bad phone numbers, UPI IDs, domains, and URLs — an instant, zero-cost match short-circuits the AI call entirely when the input matches a known scam indicator. Persistent storage uses SQLAlchemy ORM models against SQLite in development and PostgreSQL in production (Railway), with the schema written to be Postgres-portable from day one (no SQLite-only column types).

Deployment

The backend is deployed on Railway with continuous deployment from a GitHub repository — every push to the main branch triggers an automatic rebuild and redeploy. The Flutter app points at the deployed Railway URL in production and a local/LAN address during development, controlled by a single configuration flag in api_service.dart.

6 Feature Deep Dive: Digital Arrest Scam Detection

This pillar covers four distinct input modes feeding one shared detection pipeline: pasted text/SMS/WhatsApp messages, screenshots, URLs, and call transcripts.

How it works

- The user submits a screenshot, message, URL, or call transcript through the corresponding Flutter screen.
- For text and URL inputs, the backend first checks the local scam database (scam_db.py) for an instant known-scam match — if found, a result is returned immediately with no AI call, at zero marginal cost.
- If unmatched, the input is sent to Gemini (Vision for images, LLM for text) with a structured system prompt instructing the model to return only valid JSON matching a fixed schema.
- The model returns a risk score (0-100), a scam category (one of ten India-specific types), a plain-language summary, a list of discrete red flags (each with a short label and a one-sentence explanation), and actionable advice.
- The result is saved to the user's scan history and, for verified community reports, can strengthen the shared scam database over time.

Category taxonomy

Phishing · OTP Scam · Job Scam · Loan Scam · UPI Fraud · Digital Arrest Scam · Investment Scam · Romance Scam · Courier Scam · Lottery Scam · (plus Likely Safe / Unknown fallbacks).

Call transcript analysis (scoped prototype)

Real-time interception of live phone call audio would require a native Android foreground service, runtime telephony permissions, and Play Store policy review — explicitly out of scope for a cross-platform Flutter application built within a hackathon timeline. NexusShield instead implements the legitimately shippable version of this feature: the user pastes or dictates a transcript of a call they just received (or a recording they have), and a specialised prompt (CALL_SYSTEM_PROMPT in ai_engine.py) flags digital-arrest-specific patterns — authority impersonation, manufactured urgency, instructions to stay on video and not hang up, isolation from family, and pressure to transfer money or buy gift cards — using the identical structured risk_score/red_flags schema as every other analyzer. This is explicitly labeled as a prototype notice inside the app itself.

Multi-language output

The user's saved language preference (English, Tamil, Hindi, Kannada, or Telugu) is passed with every analysis request. The summary and advice fields are generated directly in the selected language; the scam_type and category fields are deliberately kept in English so the app's icon and colour-coding logic can match them reliably regardless of display language.

7 Feature Deep Dive: Counterfeit Currency Detection

The user photographs a banknote (front and/or back) through the in-app camera or gallery picker. The image is sent to Gemini Vision with a dedicated prompt (CURRENCY_SYSTEM_PROMPT / build_currency_prompt) instructing the model to assess visible authentication cues: security thread presence and position, watermark visibility, serial number formatting and consistency, micro-lettering, colour-shift ink behaviour, and overall print quality and paper texture.

The model returns a structured verdict: a boolean genuine/likely-counterfeit assessment, a confidence percentage, and a list of specific visual reasons supporting that assessment (e.g. “Security thread is clearly visible and appears correctly positioned”, “Watermark of Mahatma Gandhi and 500 can be seen against the light”). This result is logged into the same scan-history table as every other analyzer, so it appears in the user's personal analytics alongside their scam detection history.

Honesty by design
The app surfaces an explicit, permanent notice on this screen: “this uses a general-purpose vision model as a proof of concept and is not a certified currency authentication system.” This reflects a deliberate product decision to avoid overstating the reliability of a vision-model-based assessment for a task with real financial and legal consequences.

8 Feature Deep Dive: Fraud Network Intelligence

This feature answers a question no single-record scam-database lookup can: is this bad actor operating alone, or is it part of a coordinated ring? A user searches any single identifier — a phone number, UPI ID, website, device ID, or bank account — and the system returns every other identifier that has co-occurred with it across all scam reports in the database.

Traversal algorithm

The backend (intel_router.py, get_fraud_network) finds every scam report containing the searched value in any of five linkable fields (phone_number, upi_id, website, device_id, bank_account). For each matching report, every other populated linkable field on that same report becomes a node, and an edge is drawn between every pair of identifiers that co-occur on the same report. This is a depth-1 graph traversal: “this phone number was used with these UPI IDs and this device ID across N scam reports” — sufficient to surface real coordinated fraud rings without requiring a dedicated graph database for the current scale of data.

The Flutter UI renders this as a hub-and-spoke diagram: the searched identifier sits at the centre, with every linked identifier displayed below it, colour-coded by type (phone = red, UPI = orange, bank account = blue, device ID = yellow, website = green), each annotated with how many reports it appears in.

Standalone auditability script

In addition to the live in-app search, a standalone Python script (fraud_network_intel.py, built on NetworkX) demonstrates the same clustering logic against a seeded dataset, computing a per-ring severity score (based on ring size, distinct victim count, and total amount involved) and producing a visual graph image suitable for presentation to law enforcement — directly answering PS6's call for “actionable intelligence packages ... that link across jurisdictions.”

9 Feature Deep Dive: Geospatial Crime Intelligence

Every scam report submitted through the app can optionally carry a latitude, longitude, and city. The backend (get_hotspots in intel_router.py) aggregates these into map-ready hotspot points by rounding coordinates to a roughly 1.1km grid, so nearby reports in the same neighbourhood cluster into a single marker rather than overlapping exactly. Each returned hotspot includes a report count, the dominant scam category at that location, and a full category breakdown.

The Flutter client renders this using OpenStreetMap tiles (via flutter_map) in two selectable view modes: a heatmap (layered, semi-transparent circles simulating intensity without requiring a paid heatmap tile service) and a pin view (individual markers, colour-coded by severity: green/orange/red based on report count). Tapping any hotspot — in either mode — opens a detail sheet showing the exact report count and category breakdown for that location. Results are filterable by scam category (Digital Arrest Scam, UPI Fraud, OTP Scam, Phishing, Investment Scam, Job Scam, Lottery Scam, Courier Scam).

This directly answers PS6's “Geospatial Crime Pattern Intelligence” direction: a command-centre-style view for patrol prioritisation and resource deployment, built entirely on free, zero-cost mapping infrastructure (OpenStreetMap) rather than a paid mapping API, which keeps the platform free to run and scale.

10 Feature Deep Dive: Citizen Fraud Shield

This pillar is the citizen-facing layer that ties every other feature together into one coherent, always-available product.

Scam Copilot (conversational AI)

A chat interface where a citizen can describe any suspicious situation in plain language — “Bank asked me for OTP, is that safe?”, “I got a WhatsApp lottery win message” — and receive an immediate, conversational risk verdict (Safe / Suspicious / High Risk / Definite Scam) plus concrete next steps. Under the hood this reuses the same Gemini client and category taxonomy as the structured analyzers, but with a conversational system prompt (copilot_reply in ai_engine.py) that returns plain text rather than JSON, and correctly maps chat history into Gemini's user/model role format for multi-turn context.

Clipboard Monitor

Whenever NexusShield regains foreground focus, it checks the device clipboard for URLs, UPI IDs, or wallet-address-like strings and warns the user before they paste something suspicious elsewhere. True OS-level background clipboard monitoring would require a native Android service and a runtime permission not included in this build; the in-app “warn on regain-focus” behaviour is the legitimately shippable equivalent, and is disclosed as such in the Settings screen.

Family Protection

Users can link family members' accounts (by email, once that person already has a NexusShield account) and receive alerts whenever a linked member's scan crosses a configurable risk threshold. This extends protection to less digitally-literate family members without requiring them to actively report anything themselves.

Multi-language support

Five languages — English, Tamil, Hindi, Kannada, Telugu — are supported for AI-generated output across every analyzer and the copilot, extending the platform's reach beyond English-literate users, which matters significantly given who digital arrest scams disproportionately target.

11 Platform Layer

Beyond the five PS6-aligned pillars, NexusShield is built as a real, complete product rather than a collection of detection scripts. The supporting platform layer includes:

- **Auth & Security** — Email/password signup and login, Google Sign-In (Android, real OAuth flow with SHA-1-verified Firebase configuration), JWT access tokens with revocable refresh-token sessions, forgot/reset password, email verification, and device-level session management (list and individually revoke active sessions).
- **Biometric App Lock** — Optional fingerprint/PIN gate on app launch and resume, built on local_auth with a proper AppCompat-themed native prompt and lifecycle-safe re-trigger handling.
- **Analysis History** — Every scan is saved, searchable by keyword, filterable by type/category/risk, exportable as CSV or JSON, and groupable into Today / Yesterday / This Week / Earlier buckets.
- **Scam Reporting** — Citizens can submit reports (screenshot, website, phone, UPI, description, plus optional geolocation and fraud-network identifiers) which enter an admin review queue; verified reports automatically promote their indicators into the live scam database.
- **Enterprise Dashboard** — Organizations can create a workspace, invite teammates, and view aggregate scan trends and per-member risk breakdowns.
- **Admin Panel** — User management, report review and verification, live scam database management, and platform-wide analytics, gated behind an admin-only dependency.
- **PDF Incident Reports** — Any scan result can be exported as a formatted PDF report (via ReportLab) suitable for attaching to a cybercrime.gov.in complaint, including a risk-coloured summary table and a disclaimer directing the citizen to the official national helpline (1930).
- **Personal Dashboard & Analytics** — Today's scan count, a weekly trend chart, a safe/suspicious/risky breakdown, and the user's most common encountered scam type.

12 Technology Stack

Layer	Technology
Mobile client	Flutter (Dart), Provider for state management, flutter_map + OpenStreetMap for mapping, local_auth for biometrics, google_sign_in + firebase_auth for OAuth
Backend API	FastAPI (Python), Pydantic schemas, SQLAlchemy ORM, JWT (PyJWT) + passlib/bcrypt for auth
AI / ML	Google Gemini API (gemini-2.5-flash) — Vision for images/currency, LLM for text/chat, structured JSON output mode
Database	SQLite (local development), PostgreSQL (production, Railway) — schema is Postgres-portable by design
Graph analysis	Custom depth-1 traversal (in-app, live) + NetworkX-based standalone ring-detection script (offline auditability tool)
Mapping	OpenStreetMap tiles via flutter_map (zero licensing cost, no paid mapping API dependency)
PDF generation	ReportLab (incident reports), with XML-escaping of all user/AI-influenced content before rendering
Deployment	Railway (backend, continuous deployment from GitHub main branch), Android APK (debug + release signing configs)
External integrations	Google Safe Browsing, VirusTotal, AbuseIPDB, HIBP — all implemented with graceful "not_configured" fallback until API keys are supplied

13 Scope & Auditability — Live vs. Roadmap

In the interest of full auditability, every feature is labeled below by exactly what it does today, not what it is planned to do. This distinction was treated as a first-class product requirement throughout development, not an afterthought.

Live now

- Screenshot / Text / URL scam analysis via Gemini, with local scam-database pre-check
- Call transcript scam detection (digital-arrest-pattern focused)
- Counterfeit currency scanner (photo-based, Gemini Vision)
- Fraud network graph search (live, depth-1 identifier linking)
- Geospatial hotspot map (OpenStreetMap, heatmap + pin views)
- Scam Copilot conversational chat
- Email/password authentication + Google Sign-In (Android, real OAuth)
- Biometric app lock (fingerprint/PIN)
- Multi-language output across all analyzers (5 languages)
- Analysis history (search, filter, export, PDF report generation)
- Family protection, enterprise dashboard, admin panel
- Scam reporting with admin verification promoting into the live scam database

Roadmap (explicitly not live)

- Real-time in-call audio interception — requires a native Android foreground service, telephony runtime permissions, and Play Store policy review; out of scope for a cross-platform Flutter prototype
- Automated SMS scanning — requires runtime telephony permission not included in this build
- Apple Sign-In and iOS release
- Push notifications via Firebase Cloud Messaging — notifications currently work in-app via polling; the FCM integration point is stubbed and documented (push_stub.py)
- Live external threat-intel integrations (Google Safe Browsing, VirusTotal, AbuseIPDB, HIBP) — code is fully implemented and calls the real APIs the moment a key is supplied; currently returns an explicit "not_configured" status rather than faking a result
- Direct CERT-In / cybercrime.gov.in / NCRB portal integration
- Server-side verification of Google/Apple OAuth id_tokens against provider public keys — the current implementation trusts the client-supplied token's claims directly, which is acceptable for demo/prototype use but is explicitly flagged in code comments as required before any production deployment

14 Business Impact & Scalability

Business impact

• Shifts protection from reactive complaint-taking to proactive, point-of-contact detection — a citizen can check a call, message, or banknote in the moment of doubt, before money changes hands.

• One platform covers four of PS6's five suggested build areas plus a citizen-facing fifth, rather than a single-purpose tool — directly matching the organisers' framing of this as a platform problem, not a model problem.

• Verified citizen reports continuously strengthen the shared scam database, creating a compounding network effect: every genuine report makes the next citizen's instant-match check more likely to catch a known threat before any AI call is even needed.

• Multi-language support extends reach to citizens who are not English-literate — a population that is disproportionately targeted by digital arrest and OTP scams.

Technical scalability

• Backend schema is Postgres-portable from day one (currently SQLite for local development speed only).

• Stateless JWT authentication is horizontally scalable without session affinity — any number of backend instances can serve any request.

• AI calls are isolated in a single module (ai_engine.py), so the underlying model or provider can be swapped without touching router logic.

• The local scam-database pre-check reduces both AI cost and latency at scale by resolving known-bad indicators instantly, without an API call.

• Zero-cost OpenStreetMap-based mapping keeps the geospatial feature free to run and scale, with no per-request mapping API billing.

• Modular FastAPI routers mean each feature area can be deployed, monitored, and scaled independently as load patterns diverge.

15 Known Limitations & Next Steps

This section is included deliberately, in the same spirit as Section 13 — an honest account of what a production deployment would still require.

- **OAuth token verification**: Google/Apple sign-in currently trusts the client-supplied id_token's claims directly rather than verifying them server-side against the provider's public keys. This is flagged in code and must be closed before handling real user accounts at scale.
- **Currency detection is a proof of concept, not a certified system**: a general-purpose vision model gives a strong directional signal but should not be the sole basis for a legal or financial decision. A production deployment for banks/ATMs would need a purpose-trained, benchmarked computer-vision model.
- **Fraud network traversal is depth-1**: sufficient for the current data scale, but a production deployment handling nation-scale report volume would benefit from a dedicated graph database (e.g. Neo4j) for deeper multi-hop traversal and faster querying at scale.
- **Real-time call/SMS interception** remains architecturally out of reach for a cross-platform Flutter app without a native Android module and explicit Play Store policy review — this is a scoped, not forgotten, gap.
- **Push notifications** are currently delivered via in-app polling rather than true push (FCM); the integration point is built and documented but requires Firebase project credentials to go live.

16 Conclusion

NexusShield demonstrates that PS6's five suggested directions do not need to be built as five separate tools. By sharing one authentication layer, one scan-history table, one AI integration module, and one live scam database across every feature, the platform tells a single coherent story: a citizen reports or checks something suspicious, AI analyzes it against both a local known-scam database and a large language model, the result feeds a shared intelligence layer that powers the hotspot map and fraud network graph, and law enforcement or financial institutions gain a command-centre view built from that same citizen-contributed data — intelligence before mass victimisation, exactly as the problem statement asks.

Every feature described in this document is live and testable in the current build, with the explicit exception of the items listed in Section 13's roadmap column, which are clearly out of scope for the reasons stated rather than simply unfinished. We believe this combination of breadth across PS6's suggested build areas, technical depth within each feature, and transparent scope labeling represents a genuinely production-oriented response to the challenge, not a demo built purely to impress a judging panel.

Team NexusAI · Abinivesh M · Abhinivesh K · Akshaya M · Yazhini K

NexusShield — intelligence before mass victimisation.